
TOBE FE CONVENTION

AP 차세대 생산시스템 아키텍처 설계

Version 0.7

2026-01-16

AMOREPACIFIC



본 SOW 는 '아모레퍼시픽 차세대 생산시스템 아키텍처 설계' 계약을 위해 '아모레퍼시픽' 과 'SK 주식회사 AX'간 수행되는 서비스에 대한 제반 합의사항을 규정하며 상호 신의와 성실에 입각하여 본 합의 사항을 준수합니다.

본 문서는 양 당사자간의 거래에 대하여 규정하며, 양 당사자가 '아모레퍼시픽 차세대 생산시스템 아키텍처 설계'와 관련하여 '아모레퍼시픽'의 RFP 및 'SK 주식회사'의 제안서, 제안설명서 등의 내용에 우선한 효력을 가집니다.

본 문서에 명기된 내용은 향후 아모레퍼시픽과 SK 주식회사 AX 간의 상호 합의하에 변경될 수 있습니다.

승 인

[고객사]

구분	성명	일자	서명
승인자			
검토자			

[수행사]

구분	성명	일자	서명
승인자	이민환		
검토자			

제 · 개정 이력

버전	변경일자	내용	작성자
0.7	2026-01-20	0.7 작성	장종민

Index / 목차

프로젝트 네이밍 및 구조 표준 가이드 (V2.0).....	4
1.1. 기본 원칙 (CORE PRINCIPLES)	4
1.2. API 네이밍 규칙 (BACKEND CONTRACT CONSUMPTION RULE).....	4
1.2.1. API 요청/응답 DTO(Type) 네이밍.....	4
1.2.2. 쿼리 파라미터 네이밍 (Search Filters).....	4
1.3. REACT 컴포넌트 구조 및 네이밍	5
1.3.1. 계층 분류.....	5
1.3.2. Modal 네이밍.....	5
1.3.3. 파일 확장자 및 컴포넌트 분리 기준.....	5
1.4. 커스텀 Hook 네이밍	5
1.5. 이벤트 핸들러 및 기타 변수	5
1.6. 인증/권한 네이밍 및 라우트 메타	6
1.7. ZUSTAND 스토어 네이밍.....	6
1.8. CHADCN 사용 규칙	6
1.9. 폴더 구조.....	7
1.9.1. Frontend (React)	7
1.9.2. Component Terminology	7
1.9.3. API 파일명 규칙.....	8
1.10. IMPORT PATH RULES (DEPENDENCY & BOUNDARY RULES).....	8
1.10.1. 기본 원칙.....	8
1.10.2. 설계 의도.....	9
1.11. 주요 ESLINT 규칙 (LINTER RULES).....	9
1.11.1. 전역 규칙 (Global Rules).....	9
1.11.2. 페이지 전용 규칙 (src/pages/**/*.*.tsx)	9
APPENDIX A. 임시 BFF (BACKEND MOCK) 운영 정책	10
A.1 목적 (Purpose).....	10
A.2 수명 정책 (Lifecycle Policy)	10
A.3 지원 API 범위 (고정).....	10
A.4 금지된 책임.....	10
A.5 계약 권한 (Contract Authority).....	11

프로젝트 네이밍 및 구조 표준 가이드 (v2.0)

1.1. 기본 원칙 (Core Principles)

- **의도 중심 네이밍:** 이름만 보고도 객체의 역할(페이지, 기능, 상태)과 위치를 알 수 있어야 합니다.
- **레거시 격리:** 기존 시스템의 명칭이나 화면 ID 는 오직 코드 주석과 BFF 내부 릴레이 계층에서만 허용합니다.
- **축약 금지:** 모든 명칭은 풀 네임(Full Name) 사용을 원칙으로 합니다.
 - (예: **WorkOrder** (O), **WO** (X) / **Detail** (O), **Dtl** (X))
- **DTO/VO 책임 분리:** DTO 는 API 계약, 화면 VO(ViewModel)는 UI 모델로 구분하며 **혼용을 금지**합니다.

1.2. API 네이밍 규칙 (Backend Contract Consumption Rule)

프론트엔드는 API 계약을 정의하지 않으며, 백엔드가 제공하는 URL 을 그대로 소비합니다. 이 섹션은 백엔드 API 호출 시 프론트엔드 측의 일관성을 유지하기 위한 가이드입니다.

Backend API 구조는 Backend 팀의 권한이며,
프론트엔드는 이를 변경하거나 재정의하지 않습니다.

현재 합의된 방향은 다음과 같습니다.

- 업무 도메인: **/domain/{legacyNexacroCommand}/{legacyNexacroFunction}**
- 공통 영역: **/common/****
- 인증 영역: **/auth/****

1.2.1. API 요청/응답 DTO(Type) 네이밍

- **규칙:** API 관련 타입은 **단수형 + PascalCase** 를 사용하고, 접미사로 **Req, Res** 를 붙입니다.
 - 예: **WorkOrderReq, WorkOrderRes**

1.2.2. 쿼리 파라미터 네이밍 (Search Filters)

- **규칙:** 쿼리 파라미터는 **camelCase** 를 사용합니다.
 - 예: **/api/work-orders?orderStatus=OPEN&workerId=123**

1.3. React 컴포넌트 구조 및 네이밍

컴포넌트와 클래스는 개념적인 단위를 정의하므로 ****단수형(Singular)****과 **PascalCase**를 사용합니다.

1.3.1. 계층 분류

1. **Page (...Page):** `WorkOrderPage.tsx` (라우트와 1:1 매칭)
2. **View (...View):** `WorkOrderListView.tsx` (화면 전체 레이아웃)
3. **Section (...Section):** `WorkOrderSearchSection.tsx` (기능 단위 UI 블록)

1.3.2. Modal 네이밍

- 규칙: `{행위}{대상}Modal` (단수형 유지)
- 예시: `CreateLipstickModal`, `EditWorkOrderModal`, `SelectLineModal`

1.3.3. 파일 확장자 및 컴포넌트 분리 기준

- 규칙: JSX를 포함하는 컴포넌트는 `.tsx`, 순수 로직(Hook/Util)은 `.ts`를 사용합니다.
- 페이지 전용 컴포넌트: `pages/{resource}/components/`에 둡니다.
- 공통 컴포넌트: 여러 페이지에서 재사용되는 경우에만 `src/components/`에 둡니다.

1.4. 커스텀 Hook 네이밍

Hook은 해당 도메인 로직을 다루므로 ****단수형(Singular)****과 **camelCase**를 사용합니다.

1. 엔터티 상태 관리: `use{Entity}` (예: `useLipstick`)
2. 명백한 행위: `use{Action}{Entity}` (예: `useApproveWorkOrder`, `useSaveWorkOrders`)
3. 화면 전용 로직: `use{PageName}Page` (예: `useWorkOrderPage`)

1.5. 이벤트 핸들러 및 기타 변수

- 이벤트 핸들러 (Internal): 내부 함수는 `handle-` 접두사를 사용합니다.
 - 예: `const handleCommitButtonClick = () => { ... }`
- 이벤트 Props (External): 전달받는 함수 Props는 `on-` 접두사를 사용합니다.
 - 예: `<WorkOrderSection onCommit={handleSaveButtonClick} />`
- 표준 버튼 Props 명칭: 화면 유형(그리드/폼)에 관계없이 아래 이름으로 통일합니다.

- 조회: `onSearch`
 - 저장(DB 반영): `onCommit` (생성/수정/삭제를 포함하는 단일 반영)
 - 삭제(별도 버튼이 있을 때만): `onDelete`
 - 상수: `UPPER_SNAKE_CASE` (예: `MAX_PAGE_SIZE`)
 - 변수/배열: 데이터 목록인 경우 복수형을 사용합니다. (예: `workOrders`, `lipstickList`)
-

1.6. 인증/권한 네이밍 및 라우트 메타

- Role 상수: `ROLE_ADMIN`, `ROLE_MANAGER` 처럼 `ROLE_` 접두사의 `UPPER_SNAKE_CASE` 를 사용합니다.
 - 라우트 메타 키: `requiresAuth`, `requiredRoles` 를 표준 키로 사용합니다.
 - 리다이렉트 정책:
 - 로그인 없음 → `/login` 으로 이동하며 원래 경로를 `redirect` 쿼리 또는 `location.state` 로 보관합니다.
 - 권한 불일치 → `/forbidden` 으로 이동합니다.
-

1.7. Zustand 스토어 네이밍

- 스토어 훅: `use{Entity}Store` 형식을 사용합니다. (예: `useAuthStore`, `useLipstickStore`)
 - 스토어 파일명: `kebab-case` 기반의 `-store` 접미사를 사용합니다. (예: `auth-store.ts`)
-

1.8. chadcn 사용 규칙

1. `shadcn/ui` 로 생성된 컴포넌트는 반드시 `src/components/ui` 아래에 위치한다.
2. layout 성격 (`split-view`, `pane`, `layout`)은 `src/components/layout` 에만 위치한다.
3. `pages/**` 내부에는 `chadcn` 기본 컴포넌트를 직접 생성하지 않는다. (항상 `ui` 또는 `layout` 을 import)

`shadcn/ui` 의 설정 파일, 생성 절차, 운영 체크리스트는 Appendix 문서를 따른다.

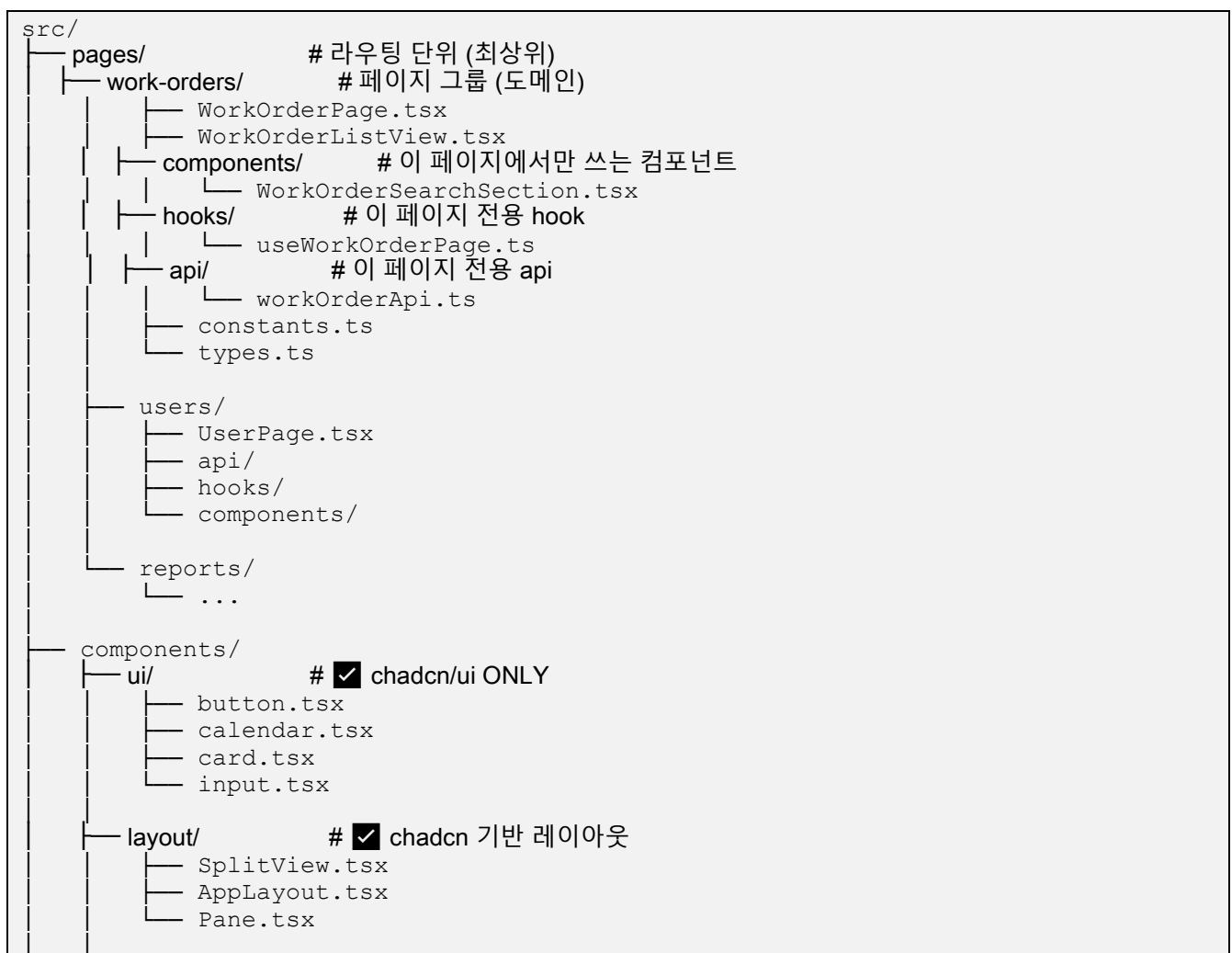
1.9. 폴더 구조

1.9.1. Frontend (React)

1.9.2. Component Terminology

1. Shared Component
 - src/components/** 아래에 위치
 - 여러 페이지에서 재사용됨
 - 반드시 `@/components/**` 로 import
2. Page Component
 - src/pages/** 내부에 위치
 - 특정 페이지(도메인)에 종속됨
 - pages 내부에서만 사용됨

도메인 단위의 폴더는 리소스의 집합이므로 **복수형(Plural) + kebab-case** 를 사용합니다.





1.9.3. API 파일명 규칙

- 공통 API 파일: camelCase + Api 접미사 사용 (예: authApi.ts, pagesApi.ts)
- HTTP 클라이언트: httpClient.ts 고정
- 금지: kebab-case 파일명 (auth-api.ts, pages-api.ts 등)

1.10. Import Path Rules (Dependency & Boundary Rules)

1.10.1. 기본 원칙

1. Shared Component / Hook / API (src/components, src/hooks, src/api 등) → 반드시 @ alias 를 사용한다.
2. Page Component / Page Hook / Page API (src/pages/** 내부) → 같은 페이지 그룹 내부에서만 상대 경로 사용을 허용한다.
3. pages → Shared 영역 접근 → 반드시 @ alias 를 사용한다.
4. Shared 영역에서 pages 접근 → 절대 금지한다.

1.10.2. 설계 의도

- 상대 경로는 “페이지 내부 결합”을 의미한다.
- @ 경로는 “공통 의존성”을 의미한다.
- import 구문만 보고도 코드의 책임 범위를 파악할 수 있어야 한다.

1.11. 주요 ESLint 규칙 (Linter Rules)

코드의 일관성과 안정성을 위해 다음과 같은 ESLint 규칙을 적용합니다.

1.11.1. 전역 규칙 (Global Rules)

- **console.* 사용 금지 (경고):** 운영 코드에 디버깅용 `console` 로그가 포함되지 않도록 경고를 표시합니다.
- **debugger 사용 금지 (에러):** `debugger` 구문이 코드에 남아있을 경우 빌드 에러를 발생시켜 의도치 않은 실행 중단을 방지합니다.

1.11.2. 페이지 전용 규칙 (src/pages/**/*.tsx)

페이지 컴포넌트는 코드 스플리팅(Code Splitting)의 기본 단위이며, 예측 가능한 로딩을 위해 아래와 같은 엄격한 규칙을 적용합니다.

- **Side Effect 호출 금지:** 페이지 파일의 최상단 스코프(Top-level scope)에서 함수를 즉시 호출할 수 없습니다. 이는 `import` 시점에 코드가 실행되어 예기치 않은 부작용을 일으키는 것을 방지하기 위함입니다.
 - (X) 잘못된 예: `const initData = fetchData();`
 - (O) 올바른 예: `useEffect(() => { const data = fetchData(); }, []);`
- **Side Effect import 금지:** 페이지 파일 내에서 전역 스타일시트(*.css)나 초기화 스크립트(init.ts 등)처럼 그 자체로 실행되는 파일을 `import` 할 수 없습니다. 이러한 Side Effect 코드는 애플리케이션의 메인 진입점(main.tsx)에서 한 번만 로드해야 합니다.
- **default export 사용 강제:** 모든 페이지 컴포넌트는 반드시 `default export` 를 사용해야 합니다. 이는 React 의 지연 로딩(Lazy Loading) API 인 `React.lazy()`와 가장 잘 호환되는 방식입니다.

- (O) 올바른 예: `export default MyPage;`
- (X) 잘못된 예: `export const MyPage = () => ...;`

Appendix A. 임시 BFF (Backend Mock) 운영 정책

본 부록은 실제 **Backend** 가 준비되기 전에만 사용되는 임시 BFF 에 대한 엄격한 규칙을 정의합니다.

A.1 목적 (Purpose)

- BFF 는 오직 임시 **Backend Mock** 으로만 존재합니다.
- 존재 목적은 다음으로 제한됩니다.
 - 프론트엔드 개발 검증
 - Nginx 프록시 구성 검증
- BFF 는 영구 아키텍처 구성 요소가 아닙니다.

A.2 수명 정책 (Lifecycle Policy)

- BFF 는 반드시 제거되거나 실제 **Backend** 로 완전히 대체되어야 합니다.
- 실제 Backend 와의 장기 공존은 명시적으로 금지됩니다.

A.3 지원 API 범위 (고정)

- BFF 는 사전에 승인된 고정된 API 집합만을 제공할 수 있습니다.
- 허용되는 최대 범위는 다음과 같습니다.
 1. 인증 관련 API (로그인, 리프레시 토큰)
 2. 전역 부트스트랩 데이터 (메뉴, 권한)
 3. UI 검증을 위한 대표 도메인 API 최대 1~2 개

새로운 API 범주 추가 또는 도메인 범위 확장은 엄격히 금지됩니다.

A.4 금지된 책임

BFF 는 다음을 절대 수행해서는 안 됩니다.

- 실제 비즈니스 규칙 구현
- 권한 판단 또는 인가 로직 수행
- 상태 저장 또는 영속성 유지
- 실제 Backend 계약에 존재하지 않는 API 정의

A.5 계약 권한 (Contract Authority)

- 실제 Backend 가 **유일한 진실 공급원(Single Source of Truth)** 입니다.
- BFF 는 계약을 **정의하지 않으며**, 반드시 Backend 계약을 **따라야만 합니다**.

